Certificate in Quantitative Finance

Final Project Brief

You are about to start the Certificate in Quantitative Finance (CQF) exam. By accessing the exam materials, you acknowledge and agree to the CQF Terms of Use (https://www.fitch.group/terms-of-use/) and the following terms:

Fitch Learning Limited ("Fitch Learning") holds ownership or licenses for all intellectual property rights related to the www.cqf.com website and any content available on the website and the exam, including text, graphics, software, photographs, images, videos, sound, trademarks, and logos (the "CQF Materials").

By clicking "Start Assignment," you consent not to infringe upon Fitch Learning's intellectual property rights. You agree not to copy, modify, reproduce, publish, sell, upload, broadcast, post, transmit, license, distribute, adapt, merge, translate, disassemble, decompile, or reverse engineer any part of the CQF Materials without Fitch Learning's written consent.

Fitch Learning reserves the right to take action against any unauthorized use of the CQF Materials in the event of a breach of these terms. This may include but is not limited to: (i) disabling and terminating your account; (ii) requesting you to remove materials that breach these terms; and/or (iii) commencing legal proceedings to protect Fitch Learning's or its licensors' intellectual property rights.

January 2026 Cohort

This document outlines topics available for this cohort. No other topics can be submitted. Each topic has by-step instructions to give you a structure (not limit) as to what and how to implement.

Marks earned will strongly depend on your coding of numerical techniques and presentation of how you explored and tested m,a quantitative model (report in PDF or HTML). Certain numerical methods are too involved or auxiliary to the model, for example, do not recode optimisation or RNs generation. Code adoption allowed if the code fully modified by yourself.

A capstone project requires own study and ability to work with documentation on packages that implement numerical methods in your coding environment e.g., Python, R, Matlab, C#, C++, Java. You do not need to pre-approve the coding language and use of libraries, including very specialised tools such as Scala, kdb+ and q. However, software like EViews is not coding.

Exclusively for current CQF delegates. No distribution.

To complete the project, you must code the model(s) and its numerical techniques form one topic from the below options and write an analytical report. If you continue from a previous cohort, please review topic description because tasks are regularly reviewed. It is not possible to submit past topics.

1. Portfolio Construction using Black-Litterman Model and Factors (PC)
2. Credit Spread for a Basket Product (CR)
3. Deep Neural Networks for Solving High Dimensional PDEs (DN)
4. Volatility Surface: Local Volatility and Optimal Hedging (LV)
5. Pairs Trading Strategy Design & Back test (TS)
6. Algorithmic Trading for Trend-Following and Reversion (AL)
7. Deep Learning for Asset Prediction (DL)
8. Blending Ensemble for Classification (ML)

Topics List for the current cohort will be available on the relevant page of Canvass Portal.

Project Report and Submission

- First recommendation: do not submit Python Notebook 'as is' – there is work to be done to transform it into an analytical report. Remove printouts of large tables/output. Write up mathematical sections (with LaTeX markup). Write up analysis and comparison for results and stress-testing (or alike). Explain your plots. Think like a quant about the computational and statistical properties: convergence/accuracy/variance and bias. Make a table of the numerical techniques you coded/utilised.
- Project Report must contain sufficient mathematical model(s), numerical methods and an adequate conclusion discussing pros and cons, further development.
- There is no set number of pages. Some delegates prefer to present multiple plots on one page for comparability, others choose more narrative style.
- It is optimal to save Python Notebook reports as HTML but do include a PDF with page numbers – for markers to refer to.
- Code must be submitted and working.

FILE 1. For our download and processing scripts to work, it is necessary to name and upload the project report as ONE file (pdf or html) with the two-letter project code, followed by your name as registered on CQF Portal.
Examples: TS John Smith REPORT.pdf or PC Xiao Wang REPORT.pdf

FILE 2. All other files, code and a pdf declaration (if not the front page) must be uploaded as additional ONE zip file, for example TS John Smith CODE.zip. In that zip include converted PDF, Python, and other code files. Do not submit unzipped .py, .cpp files as cloud anti-virus likely to flash red on our side.
Do not submit files with generic names, such as CODE.zip, FinalProject.zip, Final Project Declaration.pdf, etc. Such files will be disregarded.

Submission date for the project is Tuesday 18th August 2026, 23.59 BST

There is no extension time to Final Project.

Projects without a hand-signed declaration or working code are incomplete.

Failure to submit ONE report file and ONE zip file according to the naming instructions means such a project will miss an allocation for grading.

All projects are checked for originality. We reserve an option of a viva voce before the qualification to be awarded.

Project Support

Advanced Electives

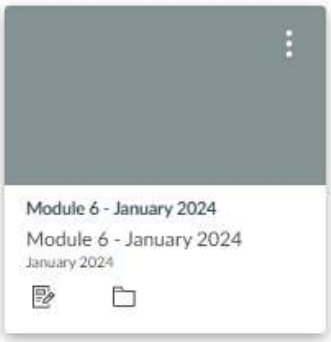
To gain background knowledge in a focused way, we ask you to review two Advanced Electives. Electives canvass knowledge areas and can be reviewed before/at the same time/closer to writing up Analysis and Discussion (explanation of your results).

- There is no immediate match between Project Topics and Electives
- Several workable combinations for each Project Topic are possible.
- One elective learning strategy is to select one `topical elective' and one `coding elective.'

To access the electives:

Login to the CQF Learning Hub

Navigate to Module 6 on your Dashboard.



Click on *Electives* button on global navigation menu.

Scroll down to *Electives*, then click the *Electives Catalog*.

You will be redirected to the electives Catalogue, where you can view and review all electives available to you. Full descriptions for each elective can be found here.

When on an elective click the *enrol* button

You will see the confirmation page, click the *enrol in Course* button to confirm your selection.

You will land on the successful enrolment page, where you can click to start the elective or return to the catalogue page.

When on the catalogue page you can click the *Learning Platform* link to return to Canvas. Your electives selected will appear on your learning dashboard.

Workshop & Tutorials

Each project title is supported by a faculty member alongside a set of project workshops and tutorials.

DATE	TITLE	TIME
04/07/2026	Final Project Workshop I (CR, PC & LV)	13:00 – 16:30 BST
11/07/2026	Final Project Workshop II (TS, DL, ML, AL, DN & LV	13:00 – 16:30 BST
06/07/2026	Final Project Tutorial I (DN & PC topic)	18:00 – 19:30 BST
09/07/2026	Final Project Tutorial II (CR topic)	18:00 – 19:30 BST
20/07/2026	Final Project Tutorial III (TS, LV & AL topic)	18:00 – 19:30 BST
23/07/2026	Final Project Tutorial IV (DL & ML topic)	18:00 – 19:30 BST

Faculty Support

Lead: Riaz Ahmad

Title: Credit Spread for a Basket Product (CR)

Project Code: CR

Lead: Kannan Singaravelu

Title: Deep Learning for Asset Prediction (DL)

Blending Ensemble for Classification (ML)

Project Code: DL & ML

Lead: Richard Diamond

Title: Pairs Trading Strategy Design & Backtest (TS)

Volatility Surface: Local Volatility and Optimal Hedging (LV)

Algorithmic Trading for Trend-Following and Reversion (AL)

Project Code: TS, LV, AL

Lead: Panos Parpas

Title: Portfolio Construction using Black-Litterman Model and Factors

Deep Neural Networks for Solving High Dimensional PDEs (DN)

Project Code: PC & DN

To ask faulty a question on your chosen topic, please submit a support ticket by clicking on the Support button which can be found in the bottom hand right corner on your portal.

Portfolio Construction using Black-Litterman Model and Factors

Construct a factor-bearing portfolio, compute at least two kinds of optimisation. Within each optimisation, utilise the Black-Litterman model to update allocations with absolute and relative views. Compute optimal allocations for three common levels of risk aversion (Trustee/Market/Kelly Investor). Implement systematic backtesting: which includes both, regressing results of your portfolio on factors and study of the factors themselves (wrt the market excess returns).

Kinds of optimisation: Mean-Variance, higher-order moments (min coskewness, max cokurtosis), max Sharpe Ratio, min VaR – implement at least two. Min Tracking Error also possible but that limits your universe and portfolio choice to a strict benchmark. Computation by ready formula or quadratic programming routine. Adding constraints improves robustness: most investors have margin constraints / limited ability to borrow / no short positions.

OPTIONALLY, compute Risk Contributions for a chosen portfolio after optimisation. ERC/Risk Budgeting approach to portfolio construction requires solving a non-linear system of budget equations, which is not strictly an optimisation but solved with sequential quadratic programming (SQP).

Portfolio Choice and Data

The choice of portfolio assets must reflect optimal diversification – according to your own understanding and approach. The simplest technical choice is choosing assets with the least correlation. For exposure/tilts to factor(s) – study factor betas *a priori* – of your assets *wrt* factor series. One recipe: include assets with either high or low beta, depending on your purpose.

A portfolio of S&P500 large caps is exposed to one factor, which is insufficient. A specialised portfolio for an industry, emerging market, credit assets – can have 5-15 names, and > 3 uncorrelated assets, such as commodity, VIX, bonds, credit spreads, real estate.

Factor series (or several) can represent your uncorrelated asset – including factors as assets at the start and giving them preferential BL views is one way of systemic implementation of factor tilting.

- MV was specified by Harry Markowitz for simple returns in excess of the r_f. USD risk-free rate is 3M US Treasury from FRED dataset.
- Most of portfolio optimisation kinds are *overly sensitive* to inputs: that principle must guide your decisions on both asset choice and statistical choices: as a guide, portfolio to have 5-15 names. Use 2-3 year sample, which means > 500 daily returns.
- The challenge with *equilibrium returns* π: not each collection of assets (investment universe) a ready benchmark index, unless it comes by design (eg, portfolio investing into an emerging market MSCI South Korea, MSCI India, or specific industry only, eg Semiconductors). The simplest approach to compute weights is market cap approach, and you can make assumptions about cap where data is unavailable. Where available, a benchmark gives you index allocations $\widetilde{w}$, into which 'an average investor is allocated'.

Step-by-Step Instructions

Part I: Factor Data and Study (Backtesting)

This part to be considered *before* and *after* the main part.

1. Portfolio Choice based on your approach to optimal diversification. The aim is to select a few assets that give the same profile of risk-adjusted returns as a much larger, naturally diversified benchmark. Q&A document distributed with the Workshop is dated but provides some guidance.
2. Experiment which factors you are going to introduce, collect their time series data. You can compute a factor yourself.

- In broad terms, simply including uncorrelated assets (to a degree, not full 0.0 correlation), which are also meaningfully unrelated in industry or investment style to the rest of assets – can be considered factor exposure (soft tilting).
- The standard experiment is with five Fama-French factors are HML (value factor), SMB (small business), RMW (robust vs. weak profitability) and CMA (conserv. vs aggressive capex).
- It recommended: use interesting and custom factors Betting Against Beta / Momentum – focus on one and relate your portfolio choice (before opt) or performance (after opt) to it.

3. Below are soft ways of introducing factors,

Universe Include factor time series or long-only factor ETFs *as investable assets* from the start, then express the tilt through BL views and/or preferential expected returns. The optimiser allocates to the factor like any other asset, subject to your constraints.

Screen Use rolling beta as a selection screen: regress candidate ETFs (eg, industry ETFs/ S&P500 sector ETFs) on your factor(s) over a rolling window and admit names whose beta is high or low as your purpose dictates. Here the factor never enters the optimiser directly.

Diagnositc Compute rolling beta of the *already-optimised* portfolio *wrt* the factors – this check realised exposure matches intent is the same as systematic backtesting.

4. Present on systematic backtesting: P&L returns and drawdowns, type of analytics and performance ratios of *pyfolio_reloaded* (you don't need to use that exact package).
5. The work on factor understanding and systematic backtesting can also be done *before* optimisation. For a selection screen: compute rolling beta on tickers (candidate assets) on your factor(s). For an understanding of factors, compute rolling beta on factors/factor ETFs vs the market. The market doesn't have to be S&P500.

Part II: Comparative Analysis of BL outputs

1. Black-Litterman application is guided by the guide by T. Idzorek. Find a ready benchmark or construct the prior: some equilibrium returns can come from a broad market index. Explicitly code computational BL formulae for the posterior returns.
2. Imposing too many views will make seeing impact of each individual view difficult.
3. Describe analytically, compute and study optimisation results for at least two kinds.
4. Adding constraints improves robustness: margin constraints / limited ability to borrow / no short positions / sector caps, $|w_i| \leq w_{\max}$, and common $1^\top w = 1$. Constraints remove known solution for w^* – you will need to compute allocations using Quadratic Programming: simply utilise a ready solver: `cvxpy`, `cvxopt`, `scipy.optimize.minimize`.
5. Moments: our goal to minimize the portfolio risk expressed by the even moments, while maximizing the odd moments, subject to constraints – min coskewness, max cokurtosis – that counts as specialised optimisation.

6. Present multiple sets of optimal allocations, even for a classic mean-variance optimisation (but inputs varied). Make your own selection on which results to focus your Analysis and Discussion – the most feasible and illustrative comparisons.
    - Optimal allocations (your) vs benchmark allocations for active risk. Expected returns vs equilibrium returns (alike to Table 6 in BL Guide). Treat expected returns μ as naïve estimate.
    - BL Views are not affected by covariance matrix – that opens a possibility to combine BL and covariance shrinkage (optional).
    - Three levels of risk aversion – recommended for classical MV.
7. There is no rebalancing task for the project: posterior BL allocations expected to be durable. You can introduce rebalancing for your own understanding.
8. Optionally, you can quickly compute and compare performance over time of one-two your chosen allocations vs $1/N$ portfolio / MDP portfolio / simple Global Min Variance. Use the tools of diversification ratio and risk contributions.
9. Post-optimisation systematic backtesting *wrt* to factor series is recommended and noted in the other section.

END OF BY-STEP

Credit Spread for a Basket Product

Price a fair spread for a CDS portfolio for 5 reference names (Basket CDS) by copula method. This opens the pricing of several instruments, 1st to default, 2nd to default and so on. The copula is the device that lets us prescribe co-movement between the names while leaving each name's marginal default time untouched; the conversion $u_i \to \tau_i$ is then just an inverse-CDF read but manually implemented. Copula requires appropriate estimation of default correlation: theoretically from historical data of CDS differences but that poses issue of too much correlation in single-name CDS (low liquidity). Initial results are histograms for uniformity check, and scatter plots for co-dependence checks – these apply to correlated RNs sampled by copula, but can also apply to data before the requisite correlation is estimated. Substantial result is sensitivity analysis by repricing.

A successful project will implement sampling from both, Gaussian and t copulae, and price all k-th to default instruments (1st to 5th). Spread convergence can require the low discrepancy sequences when sampling (eg, Halton, Sobol): such sequences fill the unit hypercube more evenly than pseudo-random draws, so the sampling error decays closer to $O(N^{-1})$ rather than the $O(N^{-1/2})$ of plain Monte Carlo. That matters for senior baskets (4th/5th-to-default), whose spread is driven by rare joint defaults in the tail – exactly where ordinary pseudo-random points are sparse. Sensitivity analysis *wrt* inputs is required.

A successful project will implement sampling from both, Gaussian and t copulae, and price all k-th to default instruments (1st to 5th). Spread convergence can require the low discrepancy sequences (e.g., Halton, Sobol) when sampling. Sensitivity analysis *wrt* inputs is required.

Data Requirements

Two **separate** datasets required, together with matching discounting curve data for each.

1. **A snapshot of credit curves** on a particular day. A debt issuer likely to have a USD/EUR CDS curve – from which a term structure of hazard rates is bootstrapped and utilised to obtain exact default times, $u_i \to \tau_i$. In absence of data, spread values for each tenor can be assumed or stripped visually from the plots in financial media. The typical credit curve is concave (positive slope), monotonically increasing for $1Y, 2Y, \ldots, 5Y$ tenors.

2. **Historical credit spreads series** taken at the most liquid tenor $5Y$ for each reference name. Therefore, for five names, one computes 5×5 default correlation matrix. But choose 5 listed companies as reference names: is much easier to compute correlation matrix from equity returns. Corporate CDS series are not in open access: they can be obtained from Bloomberg or Reuters terminals (via your firm or a colleague). For sovereign credit spreads, time series of ready bootstrapped PD_{5Y}, explore data sources such as `www.datagrapple.com` and `www.quandl.com`.

   Even if CDS_{5Y} and PD_{5Y} series are available with daily frequency, the co-movement of daily changes is market noise *more* than correlation of default events, which are rare to observe. Weekly/monthly changes give more appropriate input for default correlation, however that entails using 2-3 years of historical data given that we need at least 100 data points to estimate correlation with the degree of significance.

If access to historical credit spreads poses a problem remember, default correlation matrix can be estimated from historic equity returns or debt yields.

Step-by-Step Instructions

1. For each reference name, bootstrap implied default probabilities from quoted CDS and convert them to a term structure of hazard rates, $\tau \sim Exp(\hat{\lambda}_{1Y}, \ldots, \hat{\lambda}_{5Y})$.

2. Estimate default correlation matrices (near and rank) and d.f. parameter (ie, calibrate copulæ). You will need to implement pricing by Gaussian and t copulæseparately.

3. Using sampling form copula algorithm, repeat the following routine (simulation):

   (a) Generate a vector of correlated uniform random variable.

   (b) For each reference name, use its term structure of hazard rates to calculate exact time of default (or use semi-annual accrual).

   (c) Calculate the discounted values of premium and default legs for every instrument from *1st* to *5th*-to-default. Conduct MC separately or use one big simulated dataset.

4. Average premium and default legs across simulations separately. Calculate the fair spread.

Model Validation

- The fair spread for k*th*-to-default Basket CDS should be less than k-1 to default. Why?

- Project Report on this topic should have a section on **Risk and Sensitivity Analysis** of the fair spread *w.r.t.*

  1. default correlation among reference names: either stress-test by constant high/low correlation or ± percentage change in correlation from the actual estimated levels.
  2. credit quality of each individual name (change in credit spread, credit delta) as well as recovery rate.

  Make sure you discuss and compare sensitivities for all five instruments.

- Ensure that you explain historical sampling of default correlation matrix and copula fit (uniformity of pseudo-samples) – that is, Correlations Experiment and Distribution Fitting Experiment as will be described at the Project Workshop. Use histograms.

Copula, CDF and Tails for Market Risk

A practical tutorial on using copula to generate correlated samples is available at: `mathworks.com/help/stats/copulas-generate-correlated-samples.html` It is useful even if you don't use Matlab for coding and presents several methods. Semi-parametric CDF fitting: the interior is Gaussian kernel-smoothed – that is, the jumpy staircase of the empirical CDF is replaced by a smooth, differentiable curve by placing a small bump (kernel) at each observation, which yields well-behaved percentiles and their inverses. The two tails are modelled with a Generalised Pareto Distribution (GPD). `mathworks.com/help/econ/examples/using-extreme-value-theory-and-copulas-to-evaluate-market-risk.html`

`mathworks.com/help/stats/examples/nonparametric-estimates-of-cumulative-distribution-functions-and-their-inverses.html`

CQF Project: Deep Neural Networks for Solving High Dimensional PDEs in Quantitative Finance (DeepPDE)

OVERVIEW

There is little doubt that deep learning techniques have revolutionized certain areas of computer science such as computer vision and natural language processing. However, the impact of deep learning and other machine learning techniques has been limited in quantitative finance, especially in traditional areas of quantitative finance such as option pricing, hedging and portfolio management. These traditional areas of quantitative finance are dominated by rigorous mathematical models, and pricing frameworks such as risk-neutral evaluation. In contrast computer science applications rely on high quality empirical data. As a result we have seen deep-neural networks be used to mainly solve regression problems in order to develop fast pricing engines (e.g. a neural network is given the prices of various contracts and learns the pricing function). However in the last few years we have observed a number of approaches that go beyond the simple function fitting/regression framework and several recent works have embraced the new set of tools. The objective of this project is to investigate the use of Deep-Neural Networks for solving high-dimensional PDEs that appear in pricing and risk management applications.

METHODOLOGY

In this project you will develop a Deep Neural Network (DNN) that can solve the following class of d-dimensional Partial Differential Equations:

$$\begin{aligned} &\frac{\partial u}{\partial t} + \frac{1}{2}\text{Tr}(\sigma(t,x)\sigma(t,x)^\top \nabla^2 u(t,x)) + \nabla u(t,x)^\top \mu(t,x) + f(t,x,u(t,x),\sigma(t,x)^\top \nabla u(t,x)) = 0 \\ &u(T,x_1,\ldots,x_d) = g(x_1,\ldots,x_d) \end{aligned} \tag{1}$$

Where:

- $u(t,x)$ with $t \in \mathbb{R}_+$, and $x \in X \subset \mathbb{R}^d$ is the unknown function we need to compute, $\nabla u(t,x)$ is the gradient of the solution with respect to x and $\nabla u(t,x)$ is the Hessian (again with respect to x).
- $\sigma(t,x)$ is a known $d \times d$ matrix valued function.
- Tr denotes the trace of a matrix and T denotes a transpose.
- f is a known nonlinear function
- g is a known boundary condition.

While it is entirely possible to develop an approach to solve the general case in (1) your starting point for this project is the Black-Scholes PDE with d uncorrelated assets and with a constant volatility and risk free rate. You are encouraged to first understand the provided code for this problem before extending it to another pricing problem. Consider the following special case of (1) that can be used to price European-style contracts,

$$\begin{aligned} &\frac{\partial u}{\partial t} + \sum_{i=1}^{d}\left(\frac{1}{2}\sigma_i^2 \frac{\partial^2 u}{\partial x_i^2} x_i^2 + r\frac{\partial u}{\partial x_i} x_i\right) - ru(t,x) = 0 \\ &u(T,x_1,\ldots,x_d) = g(x_1,\ldots,x_d) \end{aligned} \tag{2}$$

Extensions to other type of contracts and other applications of the framework (e.g. for solving stochastic optimal control problems are given at the end of this document).

In this project we will focus on one of the most promising approaches to solving (2) that relies on a solving a forward-backward system of stochastic differential equations. The derivation of the equations below will also be explained in the project workshop.

$$\begin{aligned} \mathrm{d}X_t^i &= rX_t^i\mathrm{d}t + \sigma^i X_t^i \mathrm{d}W^i, \quad X_0^i = x^i,\ i = 1, \ldots, d \\ \mathrm{d}Y_t &= rY_t\mathrm{d}t + \sum_{i=1}^{d} \sigma^i X_t^i Z_t^i \mathrm{d}W^i, \quad Y_T = g(X_T^1, \ldots, X_T^d) \end{aligned} \tag{3}$$

The interpretation of (3) is as follows: X^i denotes the price of asset i and for simplicity we use a constant $r > 0$ risk free rate. Note that for simplicity we have also assumed that $\sigma^{i,j} = 0$ for $i \neq j$ and that the Brownian motions are all uncorrelated i.e. $\mathrm{cov}(\mathrm{d}W^i \mathrm{d}W^j) = 0$ for $i \neq j$. The second equation for Y is a Backward Stochastic Differential Equation. The unknowns for the BSDE are Y_t and Z_t. In terms of interpretation it can be shown that $Y_t = u_t$ and $Z_t = \nabla_x u(t, x)$. In other words Y_t is the price of the option and Z_t is the option's delta.

Since Z_t is not known it is not possible to (easily) simulate the system in (2). Instead the idea is to parameterize the solution $u(t, x) \approx u(t, x; \Theta)$ where Θ are the parameters of a neural network, and compute $Z_t \approx Z_t(\Theta) = \nabla_x u(t, x; \Theta)$. The derivative $\nabla_x u(t, x; \Theta)$ is with respect to x and can be computed with backpropagation.

Step-by-Step Instructions

Your code should have the following main steps.

1. Define a Neural Network to represent $u(t, x; \Theta)$. See the example code provided for an example of how to set this up using Pytorch.
2. Generate M Monte Carlo paths using a simple Euler scheme to discretize the dynamics in (3). Since your Euler discretization will use $Z_t(\Theta)$ instead of the correct Z_t you will need to optimize the parameters Θ
3. Use an optimizer to optimize the Neural Network parameters. Define an appropriate loss function such as the one described in [2].

A minimum working example using the Pytorch library is provided along with the specification. The minimum working example prices vanilla call options, you should modify/extend the code to solve other problems. A successful project should: be able to demonstrate an extension of the provided code to high dimensions (e.g. $d = 100$) and (optionally) apply the framework to a different product class then the one provided. Below are some ideas and references you can use to extend your code.

- The working example is based on the work from [2]. In the paper you can find some ideas on how to train the model faster and some more stable architectures that work for very high dimensions $d = 100$.
- The article in [3] is also a good introduction to the topic and has a discussion on exotic options (still in one-dimension).
- The article in [4] reviews several architectures and algorithms for solving high-dimensional pricing problems.
- The article in [1] investigates the use of deep neural networks for XVAs.
- Feel free to explore the papers above and other references and consider extending the minimum working example to applications that are of intrest to you, e.g. introduce correlation between the assets, consider dynamics that are not log-normal, price exotics or XVAs, etc.

References

[1] A. Gnoatto, A. Picarelli, and C. Reisinger. Deep xva solver: A neural network–based counterparty credit risk management framework. *SIAM Journal on Financial Mathematics*, 14(1):314–352, 2023.

[2] B. Güler, A. Laignelet, and P. Parpas. Towards robust and stable deep learning algorithms for forward backward stochastic differential equations. *33rd Conference on Neural Information Processing Systems (NeurIPS 2019), Vancouver, Canada.*, 2019.

[3] B. Hientzsch. Deep learning to solve forward-backward stochastic differential equations. *Risk Magazine, February*, 2021.

[4] J. Liang, Z. Xu, and P. Li. Deep learning-based least squares forward-backward stochastic differential equation solver for high-dimensional derivative pricing. *Quantitative Finance*, 21(8):1309–1323, 2021.

Volatility Surface: Local Volatility and Optimal Hedging

Summary

We move to a serious quant footing for implied volatility: covering its surfaces, interpolation and the generating process $\sigma(S,t)$. Options data vendors for the most liquid tickers are indicated below. As quants, we can do more than descriptive statistics on options data. We *regularize and differentiate the surface*, and from it extract $\sigma^2_{\text{loc}}(S,t)$.

The surface construction techniques will help you understand PDEs in-depth, including the results produced by Dupire's Forward PDE and how they differ from pricing under the Backward PDE framework of Black-Scholes. Forward PDE is a threshold concept and formulated in (K,T) space, though here we do not actively evolve the surface — instead, we interpolate it and read off its slope and curvature to compute local variance. Make a distinction between **forward** and **backward PDEs**, as the forward PDE is a threshold concept in this course.

This unlocks analytical derivatives ∂_T and ∂_K, which can be directly applied in hedging recipes and options book management. We can bump inputs, recompute Greeks for exotics, and analytically compute the analogue to min variance delta (MVD). We recover the risk-neutral density $f_{S_T} = \partial_{KK} C$, which characterizes the market-implied distribution of the underlying. The outcome of your project is a robust pipeline that fits a local volatility surface consistent with market vanilla prices, and exposes analytical derivatives.

Sticky-strike vs sticky-delta

Local Volatility is known as 'sticky-strike' method, while traders consider 'sticky-delta' calibrations to be more effective. Our Min Variance Delta is such sticky-delta method. We can improve LV setup and work with moneyness, which is more relevant to sticky-delta. Another 'kernel trick' we invoke is to construct surface and derivatives on total variance $w = T\,\sigma^2_{\text{imp}}(T,y)$.

Numerical techniques of this topic will be applicable in other domains. They are (a) reformulation of PDEs (and more) with variable change, (b) derivatives computation and no-arb controls and (c), industrial-grade interpolation – monotone cubic spline, hermite segments and secants for slope, Akima spline. We will also cover *Andreasen-Huge* recipe that allows computing necessary (K,T)-derivatives from option prices directly. MV Delta estimation revisited as well, for which you can optionally try recursive weighted LS or state-space filtering for stability of $\hat{a}_t, \hat{b}_t, \hat{c}_t$ coefficients.

Options Data

- **OptionsDX** — free historical options data for major tickers (SPX, SPY, TSLA, QQQ) with strikes, expiries, bid/ask, sizes, and Greeks. Good choice that provides some free End Of Day (EOD) option quotes (in 2025, the quotes for 2023).
- **Polygon.io** — commercial API with quotes and reference data; useful if you already use Polygon for equities/crypto.
- **Thetadata** — robust dataset, daily refresh and live quotes suitable for backtesting; REST docs and quick-start available.
- **Dolthub repo** — SQL/Git-style versioned options data you can query directly.

The project does not require Bloomberg/Front Office-level data but you can certainly use a prof source – once you moved to your own grids and interpolation the data should not be classed as provider's. Liquidity filtering suggestions: quotes with DTE < 14, $|\delta_{BS}| \notin [0.05, 0.9]$, non-positive bid, or zero open interest.

By-Step on Local Volatility: from quotes to grids, no-arb (Part I)

1. Start from quotes in tuples $T_i, K_j, \sigma_{ij}^{\text{mkt}}$. It is important to start building robustness even at the stage of inputs into LV model. The choice here is total variance surface $w_{ij} = T_i(\sigma_{ij}^{\text{mkt}})^2$.

2. Decide whether to work in strike K_j or log-moneyness $y_{ij} = \ln(K_j/F_{T_i})$. Regularise y-grid per term for interpolation and derivatives; you can keep the native K-grid for pricing checks and no-arb tests.

3. At the first pass, you can trust vendor's Implied Volatility (IV) and proceed with it. Later on if surface quality is insufficiently smooth,recompute IV from the cleaned prices and your own F_{T_i}, so that prices, forwards and discounting stay internally consistent for the no-arb diagnostics. *'Box Rate'* method is based on Put/Call parity and regression slope, it reads F_T, D_T straight off option prices and sidesteps dividend estimates.

4. Across skew interpolation (per fixed T_i): build a convex slice delicately, can utilise a model-driven skew parameterisation in y or do piecewise-linear prices in K with convexity constraints – to increase the number of points before computing numerical derivatives (in our language, stencils).

   - That introduces calibrated parameters but allows for a different model at each a slice per fixed T_i.
   - Keep track of mapping among grids $w(T_i, y)$, $\sigma_{\text{imp}}(T_i, K)$, and $C(T_i, K)$.
   - One practical lesson is to do *OTM-splice* (OTM puts for $y \leq 0$, OTM calls for $y > 0$) or restrict to calls and recover puts by parity. These techniques avoid the wide spreads/noise of deep ITM options that corrupt $\partial_{yy} w$.

   Butterfly convexity is a property of *prices*, not of w itself. You can map and switch to prices surface and see if the density is non-negative at each slice $\partial_{KK} C \geq 0$.

5. Across time (per fixed y): calendar monotonicity $w(T_{i+1}, y) \geq w(T_i, y)$ will be disturbed, subject to on your skew model above, likely in long-dated OTM calls. You will need to apply repairs (eg, by choosing some max$w$) for up to 30% of your slices.

   - Interpolation of total variance $w(T, y)$ is linear in T: allowing for techniques of C^1 PCHIP monotone cubic spline (with hermite segments and secants for slope), Akima spline.
   - Enforce calendar monotonicity **before** any smoothing so regularisation cannot reintroduce arbitrage.

By-Step on Local Volatility: σ_{LV} (Part II)

Derivatives are computed by finite differences and your quant prowess here is about two decisions: where to increase the order of differencing and where to invoke modified differencing formulae known as 'boundary stencils'. Re-smoothing here means clipping derivatives or other doing other modifications, if it becomes clear that derivatives are not complying.

6. Dupire ingredients are derivatives, which we evaluate over the grid – in order to ultimately compute $\sigma_{\text{loc}}^2(K, T)$. Grid $w(T_i, y)$ – here y without a subindex reminds us that grid needs regularisation. Grid choice $C(K_j, T_i)$ leads to Andreasen-Huge recipe but you need 'a smooth price surface' – even one reconstructed back from $w(T_i, y)$.

7. High-order interior stencils buy accuracy but amplify noise and you might need wide stencils, for example 5-point/$O(h^4)$ interior and with one-sided boundary and 3-point stencils for transition to end areas or as you move from surface area populated with strikes/expiries to a less-traded area. Remember that illiquid quotes are less reliable.

8. LV can be computed on stricter areas of the surface: a near-forward band such as $y \in [-0.20, 0.07]$), keeping only points where $w_T > 0$, curvature is well-behaved, and reconstructed prices stay within butterfly-arb. Compute σ_{loc} on admissible points only and control for Dupire formula divider.

9. Expect intrinsic short-maturity roughness: $\sigma^2_{\text{loc}} \propto \partial_T w$ and the $1/\Delta T$ factor magnifies any residual noise at the short end. This is an artifact of Dupire on discrete data, not a coding bug – resist cosmetic smoothing that would break exact calibration.

Validation and Uses of LV in Hedging (Part III)

Compute $\sigma^2_{\text{loc}}(K,T)$ using the formula recommended in Project Workshop. You can validate the surface by repricing the vanillas through a *backward* PDE (eg Crank–Nicolson) and run a price-convergence study in space and time, so any repricing error can be attributed to modelling/regularisation rather than discretisation.

Local Volatility implementation gives us the fruits of $\frac{\partial \sigma_{imp}}{\partial K}$ – a slope – how fast the surface tilts if you nudge strike. We will also have Vega smarter than Vega_{BS}. We can compute δ^{LV}_{MV} under the local volatility.

For comparison, Min Variance Delta δ_{MV} can be estimated via doing regressions $a + b\delta_{BS} + c\delta^2_{BS}$ in the independent part (the dependent part includes option price) – without surface fitting. You can compute sparsely, for buckets $0.1 < \delta_{BS} < 0.9$ and say two maturity buckets $1M - 2M$ and $5M - 6M$.

Hedging effectiveness is evaluated with Gain (formula in HW 2017/Workshop). You can produce a comparison study of δ_{MV} vs surfaced-derived δ^{LV}_{MV}.

- The effectiveness of δ^{LV}_{MV} is an open-ended question – you might find that surface-implied MVD fails to beat BS – Gain positive for OTM buckets, but Gain negative near ATM/ITM, and it broke down in trending / regime-shift markets (eg the March-2023 when index vol was sticky).
- Gain is dominated by the high-path-density near-forward region; per-bucket gains in the wings barely contributed to the aggregate gain – the total, so do not be misled by OTM-only improvements. Report Gain by moneyness/delta bucket *and* by maturity.

END OF BY-STEP

Pairs Trading Strategy Design & Backtest v2026

Cointegrated relationship between prices opens way to structure an arbitrage around the mean-reversion in the special residual. We are equipped with stationarity tests (ADF, KPSS, AZ), and we extend EG/Johansen Procedures with mean-reversion. It is normal to start with testing specific pair/time period from your informed guess, but implement extended coint confirmation and signal generation scheme – over multiple periods, think how to improve mean-reversion. Search for pairs done via correlation in the industry but, trading with 100%, -100% weights not conductive to both, stationarity and reversion in the residual. Asset prices must be properly tied though the Error-Correction Model (EG/Johansen Procedure).

The numerical techniques to implement: matrix form autoregression, EG Procedure (two steps), mean-reversion evaluation (theta, half-life), and optimising Z^* at least iteratively. You are encouraged to venture into multivariate cointegration (Johansen, VECM) and robustness of weights by adaptive estimation.

Signal Generation and Backtesting

- Consider economic and event-driven reasons for the cointegration to persist. Example 1: a company and its potential acquisition target are likely to form the robust cointegration pair for the period (eg MAR vs IHG). Example 2: EWA vs EWC country ETFs during the risk-on period in commodities. Be inventive beyond equity pairs, other pairs can be FX and FX vs crypto, UST/UKT yields, commodity futures, VIX futures.
- Studies that filter the market/segments for cointegration are appropriate. Quant tools in search of pairs: (1) high correlation, (2) stationarity in residual test, (3) multivariate coint outright. Each approach can be refined.
- Arb is realised by making entry/exit decisions with $\boldsymbol{\beta}_{Coint}$ as allocations. All projects should cover (a) experiment and scheme development for trading signal generation from OU process fitting and (b) risk-backtesting.
- Does cumulative P&L behave as expected for an arb trade? P&L generated by a few or many trades, what is the half-life? Maximum Drawdown and behaviour of volatility/VaR?

Step-by-Step Instructions

Part I: Pairs trade design. Preparatory cointegration analysis

1. Recode regression estimation in matrix form as an exercise. Can implement Vector Autogregression specification tests, which are (a) stability check with eigenvalues, and (b) identifying optimal lag p with AIC BIC tests. However, these apply only for structural models between stationary changes; we are not forecasting returns in this project.
2. Implement Engle-Granger procedure and run it for each pair – later you can split the dataset into several periods and present more dynamic results. For Step 1, use the Augmented DF with lag=1. For Step 2, write a short analysis, eg analyse the significance of EC-term, has it stayed the same each period.
3. We extend the procedure with 'Step 3' (not in the original Engle-Granger) mean-reversion evaluation. That allows trade design: enter on bounds $\mu_e \pm Z\sigma_{eq}$ and exit on e_t reverting to the level μ_e.

4. Do not assume $Z = 1$, implement some kind of optimisation, or simple iterative choice: vary Z upwards and downwards in increments – and produce a chart/table analysing N_{trades} for each level of Z^*. The trade-off: wider bounds give the highest P&L with low N_{trades}, however those trades are more risky as e_t deviates well-above (below) μ_e. That itself has a potential for a structural break: spread shifting to the new level.

5. Testing for structural breaks is more an art than science – mostly beyond the scope of FP study. We do have quant tools in *Cointegration Lecture* and *Lifelong Learning*. Present your thought: on how your chosen pairs cointegrated relationship might break apart.

If comfortable with R, can utilise the ready and professional multivariate analysis (package *urca*) to identify cointegrated cases. Overall, your study should consider 2-3 different pairs.

Part II: Backtesting

Below items are recommendations, you can implement all or only a part of them as suitable to your asset classes and pairs. A 2-year period is considered to be minimum backtest, subject to the realities of cointegrated situation.

6. Perform the systematic backtesting of trading strategy. Take returns from a pairs trade (already with Z^*): produce drawdown plots, rolling Sharpe Ratio. Discuss your plots. (We can omit the rolling beta on S&P500 returns and factors.)

7. Think of scikit-learn inspired backtesting: splitting train/test subsets and other cross-validation as feasible for financial time series of higher and high frequency.

8. Cointegration has the advantage of stable β'_{Coint}: the spread stays stationary over 3-6 months without weights being updated. Whether that is a realistic assumption depends on your experience in the application of cointegration. Do experiment with re-estimation of coint relationship: eg, 8 months rolling window shifting by 10-15 days. Discuss the findings.

TS Project Workshop, Cointegration Lecture and FP Tutorial are your key resources.

Algorithmic Trading for Trend-Following and Reversion v2026

Algorithmic trading – intraday execution and short-horizon alpha – is concerned with how to optimally run strategies that are deceptively simple in maths. We set aside processing of raw exchange feeds and memory, and look in-depth at order types as allowed by the exchange, time-in-force, incorrect messages/misleading data from a broker, and own data storage (15 Min and more frequent). Before moving to operational discipline tasks in Parts II and III, the first activity for this quant project is to define strategies for algorithmic execution. Part I requirement is (a) two kinds of trend-following with indicators of your choice, (b) one kind of reversion strategy (non-trivial, no simple Z-scores).

Extend the strategies from backtesting to a live-testing – in Broker API of your choice, and these scripts can be very simple, do not overload work. Code must handle exceptions and inconsistent/incorrect information received back from the broker, such as order filled when it wasn't. Real-time trading relies on loops/*crone* jobs which check for a signal. Consider coding a specific task in Rust, GoLang, C++ but limit the scope – eg, sorting messages from a broker/exchange.

Towards Robust Algotrading

Indicators. Indicator formulae don't give explicit periods for averaging / resampling interval – experiment to see what is feasible. Experiment with signals from a combination of multiple indicators.

Modify your indicators away from purely price-based: for instance, volume-weighted reduce microstructure noise. Build bars by a fixed number of trades (tick bars), a fixed volume target (volume bars), or a fixed notional target (dollar bars). Rolling windows can also be computed over *event counts* (not minutes).

Testing. Extend backtests with hypothesis testing: here you think how to integrate with strategies the ratios Deflated Sharpe; IC $= \text{corr}(f_t, r_{t+1})$ - if relevant extend to AIC/BIC; t-stat based testing on means.

Decompose slippage, see what contributes to it. Under this heading, also qualify what makes your stats unprofitable, including typically spreads, execution costs – here you can give examples.

Explore order types *per API*, time-in-force (TIF), child-order style, routing and API-provided fallbacks.

Part I: Generic Strategies Made Proprietary

Implement *your design* of core strategies – backtesting can be done Python on historical data. With caution and explanation, you can use libraries like TA-Analysis. Full description of your experiments must be given in report, e.g., the ratios over different timeframes. Also give mathematical description and temporal nuance of indicator.

1. For a trend-following type, common choices are EMA, MACD, and ADX (Average Directional Index, an oscillator). Think and design what constitutes a trading signal (eg, crossover of EMA_{20D} with EMA_{5D}). The best approach is create a signal based on three indicators, and a regime filter (eg are we in widening bands, or are we in reversion). Use formula modifications, but VWAP can be used introduced later on for execution benchmarking.

2. Simple but practical trend indicator (primarily in FX) is of the following design:

   Sample the prices at regular intervals (30 sec) for price level or average; experiment over which longer period exactly to take an average (eg, 5-minute intervals). Can use *DataFrame.resample*.

   Compute the ratio of price (or short-average) to long-term average: near 1 signals 'no trend' as short-term prices $\approx$ the long-term prices. Uptrend is signaled by the ratio above 1, and downtrend by less than 1. Give several calibrations.

[NEW] Considerations for a reversion strategy type: OU process fitting will be 'a straitjacket', and price might need a transform into another features, for which the stationarity is at least defensible. Z^* deviation from past price/moving average price is too simplistic – through you can experiment with very short horizons (eg, mins, hours).

3. Bollinger Bands is a volatility-envelope reversion indicator wrapping a rolling mean μ_t in k std dev band. Decide/experiment with: lookback n, vol bandwidth k, simple vs exponential statistics, and what constitutes a signal (eg, $\%B < 0$ as a buy, mean-touch as exit). The construction operates on a rolling window:

   Compute rolling mean μ_t and standard deviation σ_t over window n, then form upper/lower bands U_t, L_t at $\mu_t \pm k\sigma_t$. Rough calibration: $n = 20$, $k \leq 2$ on daily bars.

   Can invoke $\%B_t$ oscillator rescales price position within the band to $[0, 1]$ — equivalent to a rescaled Z-score Z_t^*. Reversion entry: $\%B_t < 0$ long, $\%B_t > 1$ short; exit at midline.

4. OPTIONAL. To improve signal generation, Hurst exponent and Variance Ratio tests are used as *regime filters*. Hurst characterizes long-memory: < 0.5 reversion, ≈ 0.5 random walk, > 0.5 persistence. VR(q) compares variance of q-period returns (5-day period) to $q\times$ the variance over 1-period returns (daily); ratio < 1 signals a reversion.

   Attend to your own nuance of Hurst estimation and preferably make it dynamic (eg, log-returns across multiple window sizes, or full R/S rescaled-range procedure). Experiment with period q in VR(q) computation.

   If reversion regime confirmed by Steps 1-2 above then, for example, enter Bollinger/Z^* trades only when rolling $H_t < 0.5 - \delta$ or $\text{VR}_t(q) < 1 - \epsilon$. Calibration tradeoff: short estimation window is responsive but noisy; long window is stable in the sense of confirming reversion/no reversion but these essentially statistical tests lag after the returns behaviour shifts.

Part II: API and Coding Specifics

Describe order types suitable to your tickers and strategies. Present code for order handling and loops using a Broker API/work with socket/potential work with exchange-broadcasted messages. Set price parameters far from the market to avoid execution.

1. Historical backtesting of a strategy can be made on data retrieved from local files. However, here are aspects to make your project more industrial-scale:

   - Code routines checking data integrity (when receiving quotes, data from outside), and comparing to internally-stored and/or recently received data.
   - Calling data from a database/time-series database (TimescaleDB, ClickHouse, kdb+ is the legacy industry choice).
   - Strategy operating over data bars rather than price quotes – by a fixed number of trades (tick bars), a fixed volume target (volume bars), or a fixed notional target (dollar bars). Example: IBKR data is aggregated to *5-second bars*, not true tick data.

2. Common API choice is REST (Representational State Transfer), and even for that you want to run a dedicated Gateway rather than interacting with broker desktop software (eg, 500ms delays can come from Java GUI)
    - Alpaca, IB, and Oanda have their own versions. Alpaca REST API is free and includes async events handling based on WebSocket and Server Side Events (SSE).
    - REST is HTTP API that utilises methods GET (retrieve data), POST (create new data), PUT (update data). Useable for non time-critical operations such as retrieving historical data, account information, placing orders, and getting order status.
3. The industrial strength and lower latency API choice is FIX; can get familiar with messages via *simpleFIX* Python package, but there is no socket management/networking.
4. Python is fine for backtests, and trading 15-sec prices/above (even tick prices); but desks read feeds with multi-message-per- going into μs scale. Exchange-colocated firms use **Rust**, **C++**, and **Go**. OPTIONALLY, you can implement one specific, standalone task in Rust – eg, quotes sorting, order submission. Scripting languages like *NinjaTrader/Indie/MQL5 Pine Script/thinkScript* can be a component of project work.

Part III: Operational Discipline

While the market risk can be managed with quant methods like rolling VaR, order-handling risk is more important and reliant on API choices (Part II) as well as dev environment and stack of libraries choices.

1. Containerise and consider docker and crone-style scheduling (think how different parts of your code can be scheduled to run as an independent scripts).
2. Consider implications for running the cloud image of your code on a virtual server for trading order loops, MB and account data receipt and verification, and even quick life-testing (comparison to pre-backtested results when your algo should detect that a trading strategy/developed signal is going outside of its parameters).
3. Positions Tracking. Code can request account updates (eg, in loop) and provide a secondary confirmation layer.
    - Code must have a **verification** of server responses. Orders may be partially filled and/or returned with an incorrect fill information (eg, ticker not bought when it was actually bought).
    - Catch the stale websockets and connection issues.
4. In your report, have a section or chapter on execution benchmarking and performance reporting. Systematic portfolio/ reallocation is of less utility to algo. But for your learning, review *Pyfolio Reloaded*-style backtesting, VaR and Drawdown analysis (dynamic, not one figure), Beta-to-SPY and other systematic strat scorecards. OPTIONAL you can do a live replay of strategy for N trading days; or market order book (OB) reconstruction – limited in scope; and/or just save fills and $P\&L$ in your own database – to have this component and skill.

END OF BY-STEP

Deep Learning for Asset Prediction

Summary

Trend prediction has drawn a lot of research for many decades using both statistical and computing approaches, including machine learning techniques. Trend prediction is valuable for investment management as accurate prediction could ensure asset managers outperform the market. Trend prediction remains a challenging task due to the semi-strong form of market efficiency, the high noise-to-signal ratio, and the multitude of factors that affect asset prices including, but not limited to, the stochastic nature of the underlying instruments. However, sequential financial time series can be modelled effectively using sequence modelling approaches like a recurrent neural network.

Objective

Your objective is to produce a model to predict positive moves (up trend) using Long Short-Term Memory Networks. Your proposed solution should be comprehensive, with a detailed model architecture, evaluated with a backtest applied to a trading strategy.

- Choose one ticker of your interest from the index, equity, ETF, crypto token, or commodity.
- Predict trend only, for a short-term return (example: daily, 6 hours). Limit prediction to binomial classification: the dependent variable is best labelled $[0, 1]$. Avoid using $[-1, 1]$ as class labels.
- Analysis should be comprehensive, with detailed feature engineering , data pre-processing, model building, and evaluation.

Note: You are free to make such study design choices that make the task achievable. You may redefine the task and predict the momentum sign (vs return sign) or the direction of volatility. Limit your exploration to ONLY one asset. At each step, the process followed should be expanded and explained in detail. Merely presenting Python code files without a proper explanation shall not be accepted. The report should present the study in a detailed manner with a proper conclusion. Code reproducibility is a must, and the use of modular programming approaches is recommended. Under this topic, you do not recode existing indicators, libraries, or the optimisation to compute neural network weights and biases.

Step-by-Step Instructions

1. The problem statement should be explicitly specified without any ambiguity, including the selection of underlying assets, datasets, timeframe, and frequency of data used.

    - If predicting short-term return signs (for the daily move), then training and testing over up to 5 years should be sufficient. If you attempt the prediction of 5D, 10D return for equity or 1W, 1M for the Fama-French factor, you will have to increase the data required to at least 10 years.

2. Perform exhaustive Feature Engineering (FE).

    - FE should be detailed, including the listing of derived features and the specification of the target/label. Devise your approach on how to categorise extremely small, near-zero returns (drop from the training sample or group with positive/negative returns). The threshold will strongly depend on your ticker. *Example:* small positive returns below 0.25%.
    - Class imbalances should be addressed – either through model parameters or via label definition.
    - Use of features from cointegrated pairs and across assets is permitted but should be tactical. There is no single recommended set of features for all assets; however, the initial feature set should be sufficiently large. Advanced technical indicators including volatility estimators, and volume information can be a predictor for price direction.
    - **OPTIONAL** Use of news data (heatmap) and alternative data (FX, CDS, financial ratios, factors) has to be adapted to how neural nets process data sequences. Alternative features have to be sourced from your own access or a professional subscription. Transformers outperform LSTMs because they are typically applied in a setting with vastly more data, eg Level-II *order-book* vs daily bars.

3. Conduct a detailed Exploratory Data Analysis (EDA).

    - EDA helps in dimensionality reduction via a better understanding of relationships between features, uncovers underlying structure, and invites detection/explanation of the outliers. The choice of feature scaling techniques should be determined by EDA.

4. Proper handling of data is a must. The use of a different set of features, lookback length, and datasets warrants cleaning and/or imputation.

5. Feature transformation should be applied based on EDA.

    - Multi-collinearity analysis should be performed among predictors.
    - Multi-scatter plots presenting relationships among features are always a good idea.
    - Large feature sets (including repeated kinds and different lookbacks) warrant a reduction in dimensionality (number of features). Apply Self Organizing Maps (SOM), K-Means clustering, or other methods, while avoiding using Principal Component Analysis (PCA) because our prediction target is in non-linear dependence on features.

6. Perform extensive and exhaustive model building.

    - Design and describe your approach to the neural network architecture after study of RNN/LSTM properties and some initial experimentation.
    - The best model should be presented only after performing the hyperparameter optimisation. Comparison an optimised baseline model is necessary.
    - Hyperparameters to be optimised for the best model are design choices. Use experiment trackers like MLFlow or TensorBoard to present your study.

7. The performance should be evaluated using multiple metrics, including back-testing of the predicted signal applied in the form of a trading strategy.

    - Investigate the prediction quality. The standard classification report include AUC-ROC, confusion matrix, and balanced accuracy.
    - Predicted signals should be evaluated by applying them to a trading strategy.
    - Some form of P&L analysis must be present.

* * *

Blending Ensemble for Classification

Trend prediction is valuable for investment management as accurate prediction could ensure asset managers outperform the market. Trend predictions can be modelled effectively using machine learning algorithms; however, the choice of learning techniques to be adopted remains a challenging task. Ensemble learning is a powerful machine learning algorithm that combines the predictions of multiple machine learning models by mitigating the errors or biases that may exist in individual models, leveraging the collective intelligence of multiple models that leads to a more precise prediction.

Objective

Your objective is to produce a model to predict positive moves (up trend) using the Blending Ensemble technique. Your proposed solution should be comprehensive, with the detailed model architecture, evaluated with a backtest applied to a trading strategy.

- Choose one ticker of your interest from the index, equity, ETF, crypto token, or commodity.
- Predict trend only, for a short-term return (example: daily, 6 hours). Limit prediction to binomial classification: the dependent variable is best labelled $[0, 1]$. Avoid using $[-1, 1]$ as class labels.
- Analysis should be comprehensive with detailed feature engineering , data pre-processing, model building, and evaluation.
- Use only machine learning algorithms for this project.

Note: You are free to make study design choices to make the task achievable. You may redefine the task and predict the momentum sign (*vs* return sign) or direction of volatility. Limit your exploration to ONLY one asset. At each step, the process followed should be expanded and explained in detail. Merely presenting Python codes without a proper explanation shall not be accepted. The report should present the study in a detailed manner with a proper conclusion. Code reproducibility is a must and the use of modular programming approaches is recommended. Under this topic, you do not recode existing indicators, libraries, or optimisation algorithms.

Step-by-Step Instructions

1. The problem statement should be explicitly specified without any ambiguity, including the selection of underlying asset, timeframe, and frequency of data used.

    - If predicting short-term return signs (for the daily move), then training and testing over up to 5 years should be sufficient. If you attempt the prediction of 5D, 10D return for equity or 1W, 1M for the Fama-French factor, you will have to increase the data required to at least 10 years.

2. Perform exhaustive Feature Engineering (FE).

    - FE should be detailed, including the listing of derived features and specification of the target/label. Devise your approach on how to categorise near-zero returns (drop from the training sample or group with positive/negative returns). The threshold will strongly depend on your ticker. *Example:* small positive returns below 0.25%.

- Class imbalances should be addressed – through model parameters OR label definition.
- Use of features from cointegrated pairs and across assets is permitted but be tactical about design. There is no single recommended set of features; however, the initial feature set should be sufficiently large. Financial ratios, advanced technical indicators including volatility estimators, and volume information can be a predictor for price direction.
- **OPTIONAL** Use of news heatmap, credit spreads (CDS), historical data for financial ratios, history of dividends, purchases/disposals by key stakeholders, or factor data can enhance your prediction. Alternative features have to be sourced from your own access or a professional subscription.

3. Conduct a detailed Exploratory Data Analysis (EDA).
    - EDA helps in dimensionality reduction via a better understanding of relationships between features and uncovers underlying structure, and invites detection/explanation of the outliers. The choice of feature scaling techniques should be determined by EDA.
4. Proper handling of data is a must. The use of a different set of features, lookback length, and datasets warrants cleaning and/or imputation.
5. Feature transformation should be applied based on EDA.
    - Multi-collinearity analysis should be performed among predictors.
    - Multi-scatter plots presenting relationships among features are always a good idea.
    - Large feature sets (including repeated kinds and different lookbacks) warrant a reduction in dimensionality (number of features). Apply Self Organizing Maps (SOM), K-Means clustering, or other methods, while avoiding using Principal Component Analysis (PCA) because our prediction target is in non-linear dependence on features.
6. Perform extensive and exhaustive model building - make your design choices:
    - The architecture of a stacking model should involve at least three base learners.
    - Hyperparameters of each base learner should be optimised.
    - Present well on base learners and meta model, including mathematical aspects.
7. The performance of your proposed classifier should be evaluated using multiple metrics, including back-testing of the predicted signal applied in the form of a trading strategy.
    - Investigate the prediction quality. The standard classification report include AUC-ROC, confusion matrix, and balanced accuracy.
    - Predicted signals should be evaluated by applying them to a trading strategy.
    - Some form of P&L analysis must be present.

* * *

Coding for Quant Finance

- Choose programming environment that has appropriate strengths and facilities to implement the topic (pricing model). Common choice is Python, Java, C++, R, Matlab. Exercise judgement as a quant: which language has libraries to allow you to code faster, validate easier.

- Use of R/Matlab/Mathematica is encouraged. Often there a specific library in Matlab/R gives fast solution for specific models in robust covariance matrix/cointegration analysis tasks.

- Project Brief give links to nice demonstrations in Matlab, and Webex sessions demonstrate Python notebooks {does not mean your project to be based on that ready code.

- Python with pandas, matplotlib, sklearn, and tensorow forms a considerable challenge to Matlab, even for visualization. Matlab plots editor is clunky, and it is not that difficult to learn various plots in Python.

- 'Scripted solution' means the ready functionality from toolboxes and libraries is called, but the amount of own coding of numerical methods is minimal or non-existent. This particularly applies to Matlab/R.

- Projects done using Excel spreadsheet functions only are not robust, notoriously slow and do not give understanding of the underlying numerical methods. CQF-supplied Excel spreadsheets are a starting point and help to validate results but coding of numerical techniques/use of industry code libraries is expected.

- The aim of the project is to enable you to code numerical methods and develop model prototypes in a production environment. Spreadsheets-only or scripted solutions are below the expected standard for completion of the project.

- What should I code? Delegates are expected to re-code numerical methods that are central to the model and exercise judgement in identifying them. Balanced use of libraries is at own discretion as a quant.

- Produce a small table in report that lists methods you implemented/adjusted. If using ready functions/borrowed code for a technique, indicate this and describe the limitations of numerical method implemented in that code/standard library.

- It is up to delegates to develop their own test cases, sensibility checks and validation. It is normal to observe irregularities when the model is implemented on real life data. If in doubt, reflect on the issue in the project report.

- The code must be thoroughly tested and well-documented: each function must be described, and comments must be used. Provide instructions on how to run the code.

FP Topic	**Relevant Electives & Masterclasses**
Credit Spread for a Basket Product	• Counterparty Credit Risk Modeling • Numerical Methods • Advanced Risk Management (Masterclass)
Volatility Surface: Local Volatility and Optimal Hedging	• Advanced Volatility Modeling • Numerical Methods
Portfolio Construction using Black–Litterman	• Risk Budgeting for Asset Allocation • Advanced Portfolio Management • Behavioral Finance for Quants
Pairs Trading Strategy Design & Backtest	• Energy Trading • R for Data Science and Machine Learning
Deep Learning for Financial Time Series	• Advanced Machine Learning I (LSTM/Keras) • Advanced Machine Learning II (Classification)
Blending Ensemble for Classification	• Advanced Ensemble Modeling • Advanced Machine Learning II (Classification) • R for Data Science and Machine Learning
Algorithmic Trading for Reversion and Trend-Follow	• Algorithmic Trading II (Docker-Alpaca-Binance-RSI) • Decentralized Finance Technologies • Modelling using C++
DNNs for Solving High-Dimensional PDEs	• Advanced Machine Learning I (LSTM/Keras) • Advanced Volatility Modeling • Practical Machine Learning Case Studies (M5)

Table 1: Electives and Masterclasses relevance mapped to FP Topics (v2026)

Reading List for Final Project v2026

These are curated readings for each topic. Papers give a scheme/focus on either pricing model or techniques/statistical tests Whenever possible, the curated readings will be distributed with Project Workshops / Tutorials in files *Topic XX - Additional Material.zip*. Otherwise please check SSRN and your alternative sources (e.g., might not be possible for the program to distribute a particular good reading).

Reading List: Local Volatility. Optimal Hedging (LV, DH)

- FP Workshop will cover local volatility computation, starting from Forward PDE. Workshop and Tutorial will give further technical guidance on 'quality control' for the construction of surfaces.
  - Exercises/Solutions in 3.10 Advanced Volatility Modelling – issued 2025-Oct [NEW].
  - Python Labs *Finite Difference Methods* and *Implied Volatility* – exact titles can vary.
- *Optimal Delta Hedging for Options* by John Hull & Alan White. *Journal of Banking and Finance*, Sep 2017, `papers.ssrn.com/sol3/papers.cfm?abstract_id=2658343` – this is relevant to MVD implementation.
- *Lectures 6/7: Local Volatility* by Emmanuel Derman (2008) – these lecture slides cover the concept but not as nuanced to guide through the derivation.
- *The Volatility Surface: A Practitioner's Guide* by Jim Gatheral (2012) a good brief paper-based book [not distributed].

Reading List: Cointegrated Pairs (TS)

- Cointegration Lecture introduces multiple tests for stationarity, each to their own purpose. ADF is simple but exercise care about critical values from software. *Critical Values for Cointegration Tests* by J MacKinnon (2010) explains the compute; there is an additional note on *Mathematics of Dickey Fuller Test.*
  - *Tutorial: Nonstationarity* which is a useful R notebook.
  - *Tutorial: Cointegration in Rates - Multivarate Cointegration* another up to date R notebook (2023).
  - *Efficient Pair Selection for Pair-Trading Strategies* by P. McSharry applies ADF test to filter and search for coint pairs in the market.
- *Explaining Cointegration Analysis: Part I* by D. Hendry & K. Juselius can be read instead of an econometrics textbook. Part II concerns with VECM detail and deterministic terms.
- *Learning and Trusting Cointegration in Statistical Arbitrage* by R. Diamond (2014), *WILMOTT* go directly to Appendix on the OU process maths, and Appendix on number of trades.

Reading List: Credit Portfolio (CR)

- *CDO & Copula Lecture* material is for reference, slides 48-52 illustrate Elliptical copula densities and Cholesky factorisation.
- *Sampling From Copula* algorithm is in *Project Workshop.* See Ch 5 of *Monte Carlo Methods in Finance* Peter Jaekel (2002) for a schema as well.
- There is a special note on *Topic CR Q&A*, which includes Rank Correlation.

Reading List: Portfolio Construction (PC)

- CQF Lecture on *Fundamentals of Optimization and Application to Portfolio Selection*
- *A Step-by-step Guide to The Black-Litterman Model* by Thomas Idzorek, 2002 gives the schema to the BL model implementation.
- *The Black-Litterman Approach: Original Model and Extensions* Attilio Meucci, 2010. `http://ssrn.com/abstract=1117574`

Reading List: Deep Learning and Ensemble Learning (DL, ML)

- *Short-term stock market price trend prediction using a comprehensive deep learning system* by Jingyi Shen and M. Omair Shafiq, Journal of Big Data, Vol 7 (2020).
- *A graph-based CNN-LSTM stock price prediction algorithm with leading indicators* by Jimmy Ming-Tai Wu et al. (2021).
- *A comprehensive evaluation of ensemble learning for stock-market prediction* by Isaac Kofi Nti et al., Journal of Big Data, Vol 7 (2020).
- *A Blending Ensemble Learning Model for Crude Oil Price Prediction* by Mahmudul Hasan et al. (2022). `papers.ssrn.com/sol3/papers.cfm?abstract_id=4153206`.

Reading List: Solving High-Dimensional PDEs (DN)

- B. Hientzsch. *Deep learning to solve forward-backward stochastic differential equations.* Risk Magazine, February, 2021.
- J. Liang, Z. Xu, and P. Li. *Deep learning-based least squares forward-backward stochastic differential equation solver for high-dimensional derivative pricing. Quantitative Finance*, 21(8):1309-1323, 2021.

Reading List: Algorithmic Trading (AL)

- *Algorithmic Trading Workshop Notes* apply to both sessions, but things have evolved since October 2023 workshop delivery.
- *Algo Trading I* and *Algo Trading II* give Python code examples – *alpaca*, *backtesting*, *timescale* still relevant, but *openbb* access changed.
- *Data Sources for Options, Equities and Factors* tutorial useful updates (name can vary depending on cohort). You can also reuse Python Labs that work with Time Series.
- F. Barez, P. Bilokon, A. Gervais, N. Lisitsyn *Exploring the Advantages of Transformers for High-Frequency Trading*, February 2023, available at `https://arxiv.org/abs/2302.13850`.